Compte rendu de TP
Qualité et organisation d'un projet IA
SOLID – Ruff – Black

Module : Déploiement des Systèmes Intelligents

October 27, 2026

1 Objectif

Transformer un prototype de prédiction du risque de crédit, volontairement mal organisé (un seul fichier `main.py`), en un projet Python structuré, modulaire, testé et conforme aux outils de qualité Ruff et Black.

2 Exercice 1 – Diagnostic du projet initial

Problèmes identifiés :

1. **Fichiers** : données, modèle, notebook et code mélangés à la racine.
2. **Nommage** : `d`, `X`, `y`, `m`, `run` sans signification.
3. **Indentation/structure** : une fonction monolithique, étapes non séparées.
4. **Espaces** : pas d'espaces autour de `=` et des virgules, lignes vides manquantes (PEP 8).
5. **Imports** : `import os` inutilisé, imports non triés.
6. **Fonction `run()`** : charge, nettoie, sépare, entraîne, évalue et affiche (6 responsabilités).
7. **Séparation des traitements** : aucune étape réutilisable seule.
8. **Testabilité** : chemin en dur, `print` au lieu de `return`.
9. **Documentation** : aucune docstring, aucun README.
10. **Maintenabilité** : exécution à l'import, pas de `random_state`, score sur les données d'entraînement (trompeur), modèle non sauvegardé.

Un programme peut-il fonctionner tout en étant de mauvaise qualité ? Oui. Fonctionner signifie produire un résultat ; la qualité logicielle concerne la lisibilité, la testabilité, l'évolutivité et la reproductibilité. Le script initial tourne, mais il est impossible à tester, à faire évoluer ou à partager sans risque.

3 Exercice 2 – Structuration

```
credit_project/
|-- data/raw/credit.csv
|-- notebooks/exploration.ipynb
|-- models/
|-- src/credit_risk/
| |-- __init__.py |-- __main__.py
| |-- data.py |-- preprocessing.py
| |-- trainers.py |-- training.py
| |-- evaluation.py |-- prediction.py
| `-- pipeline.py
|-- tests/test_credit_risk.py
|-- docs/
|-- pyproject.toml README.md .gitignore requirements.txt
```

- **Notebooks / `src`** : le notebook sert à explorer ; le code réutilisable et testable vit dans `src`.
- **Données / code** : rythmes d'évolution différents, données volumineuses ou sensibles, non versionnées comme le code.
- **Dossier `tests`** : vérifier automatiquement le code et détecter les régressions.

4 Exercice 3 – Modularisation et SRP

Module	Responsabilité unique
`data.py`	charger le CSV
`preprocessing.py`	nettoyer, séparer features/cible
`training.py`	entraîner via un trainer
`evaluation.py`	évaluer le modèle
`prediction.py`	prédire, sauvegarder/recharger
`pipeline.py`	orchestrer les étapes

Dans `main.py`, `run()` avait six raisons de changer. Maintenant chaque module n'en a qu'une : un changement de format de données ne touche que `data.py`.

5 Exercice 4 – OCP et DIP

Dans `trainers.py`, la classe abstraite `ModelTrainer` impose la méthode `train()`, implémentée par `RandomForestTrainer`, `LogisticRegressionTrainer` et `XGBoostTrainer` (bonus).

```
class ModelTrainer(ABC):
    @abstractmethod
    def train(self, features, target):
        """Entraine et retourne un modele."""
```

- **Facilité d'ajout** : un nouveau modèle = une nouvelle classe, sans `if/elif` et sans modifier le code existant.
- **SRP** : un module = une responsabilité.
- **OCP** : extension par ajout de classes ; `training.py` et `pipeline.py` restent inchangés.
- **DIP** : `train_model()` dépend de l'abstraction `ModelTrainer`, pas de `RandomForestClassifier`.

6 Exercice 5 – Analyse avec Ruff

- Projet initial : **2 problèmes** (`F401` import inutilisé, `I001` imports non triés), corrigeables avec `ruff check -fix .`.
- Projet final : **0 problème**.
- Reste manuel en général : noms peu clairs, fonctions trop longues, logique erronée, absence de tests.

Pourquoi Ruff ne suffit-il pas ? C'est une analyse statique de règles syntaxiques et stylistiques : il ne vérifie ni la logique métier, ni l'architecture, ni les tests, ni la qualité du modèle (fuite de données, sur-apprentissage).

7 Exercice 6 – Formatage avec Black

- Avant : `black -check .` indique que `main.py` doit être reformaté.
- Après `black .` : tous les fichiers sont conformes ; `ruff check .` ne signale rien.

Ruff vs Black. Ruff *détecte* des erreurs et mauvaises pratiques ; Black *reformate* uniquement la présentation sans changer le comportement. Ils sont complémentaires.

Outil	Rôle	Exemple
PEP 8	Guide de style	Définir les conventions
Ruff	Linting	Imports inutilisés, erreurs
Black	Formatage	Espaces, présentation
SonarQube	Qualité globale	Complexité, duplication

8 Exercice 7 – Configuration `pyproject.toml`

```
[tool.black]
line-length = 88
[tool.ruff]
line-length = 88
[tool.ruff.lint]
select = ["E4", "E7", "E9", "F"]
```

1. **Intérêt** : la configuration est versionnée avec le projet, identique pour tous.
2. **Nouveau développeur** : après clonage, les outils appliquent automatiquement les mêmes règles.
3. **Reproductibilité** : mêmes règles, mêmes résultats sur toutes les machines et en intégration continue.

9 Comparaison avant / après

Critère	Avant	Après
Organisation	4 fichiers en vrac	arborescence standard
Responsabilités	1 fonction = 6 tâches	1 module = 1 rôle
Extension	modifier l'existant	ajouter une classe
Tests	0	5 tests pytest réussis
Ruff	2 problèmes	0
Black	à reformater	conforme
Évaluation	données d'entraînement	train/test split
Reproductibilité	aucune graine	`random_state=42`

10 Conclusion

La restructuration rend le projet lisible, modulaire, testable et reproductible. Les principes SOLID facilitent l'ajout de nouveaux modèles, tandis que Ruff et Black automatisent le contrôle de la qualité du code.